

// EduDiscount Platform • Technical Reference • 2026

Technical Deep Dive

Architecture, APIs, Database Schema, Security Model, and
Deployment Guide

Node.js 20

Express 4

PostgreSQL 15

Prisma ORM

Redis 7

React 18 + Vite

Tailwind CSS

Docker Compose

Nginx

JWT Auth

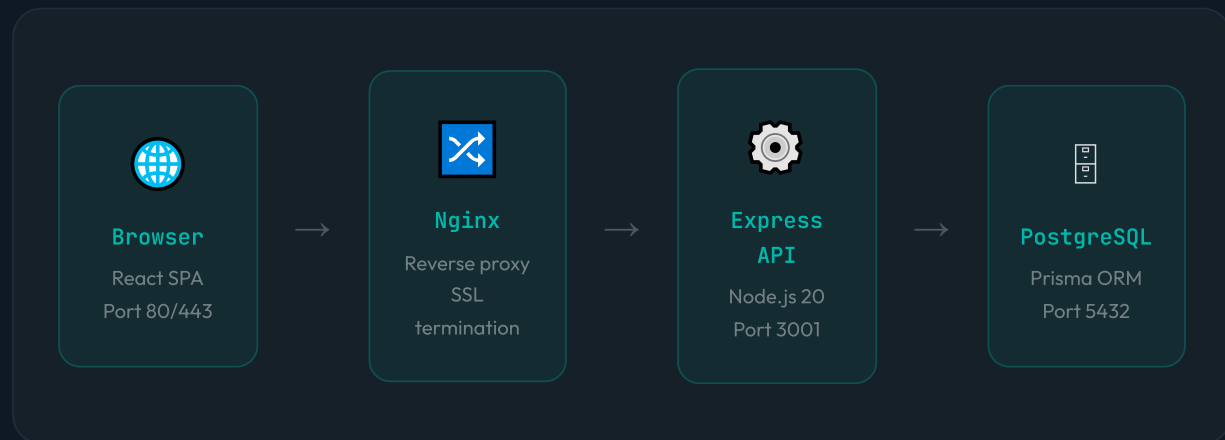
Nodemailer

Framer Motion

VERSION	SERVICES	DATABASE	API ENDPOINTS	LANGUAGES
1.0.4	5 Docker containers	6 Prisma models	30+ routes	AR + EN (i18n)

System Architecture

A microservices-style architecture running in Docker Compose. All external traffic enters through Nginx, which routes API calls to the Node.js backend and all other requests to the static React frontend.



docker-compose.yml – service topology

```

services:
  nginx:      # Entry point – routes /api → backend, / → frontend
    ports: ["80:80", "443:443"]
    depends_on: [backend, frontend]

  frontend:  # Vite build served by nginx:alpine (static)
    build: ./frontend

  backend:   # Express API – runs migrate + seed on startup
    build: ./backend
    environment: DATABASE_URL, REDIS_URL, JWT_SECRET, SMTP_*
    volumes: [uploads_data:/app/uploads]
    depends_on: [postgres, redis]

  postgres:  # PostgreSQL 15-alpine with named volume
    image: postgres:15-alpine
    volumes: [pg_data:/var/lib/postgresql/data]

  redis:     # Redis 7-alpine for code TTL caching
    image: redis:7-alpine

volumes:

```

```
pg_data:      # Persistent database storage
uploads_data: # Persistent uploaded logo storage
```

Database Schema

PostgreSQL managed via Prisma ORM. 6 models with cascade relationships. Migrations are applied automatically on container startup via `prisma migrate deploy`.

model AdminUser

id	String	@id @default(cuid())
email	String	@unique
passwordHash	String	bcrypt · cost 10
createdAt	DateTime	@default(now())

model School

id	String	@id @default(cuid())
name	String	@unique
nameAr	String?	Arabic name
accessCode	String?	school-level gate
isActive	Boolean	@default(true)
subjects	Subject[]	cascade delete
studentCodes	StudentCode[]	cascade delete

model StudentCode

id	String	@id @default(cuid())
schoolId	String	FK → School
code	String	@unique · STU-XXXXXX
label	String?	student name/ID
maxUses	Int	@default(10)
useCount	Int	@default(0)
isActive	Boolean	@default(true)

model Teacher

id	String	@id @default(cuid())
subjectId	String	FK → Subject
coursePrice	Int	piastres (÷100 = EGP)
discountPct	Int	e.g. 25 = 25% off
platformTeacherId	String	external ID
codes	DiscountCode[]	

model DiscountCode

id	String	@id @default(cuid())
----	--------	-------------------------

model Setting

key	String	@id — e.g. "logoUrl"
-----	--------	----------------------

code	String	@Unique + EDU- XXX-YYYY-ZZZZ
studentEmail	String	
originalPrice	Int	snapshot at generation
discountPct	Int	snapshot at generation
finalPrice	Int	pre-calculated
status	String	ACTIVE USED EXPIRED
expiresAt	DateTime	now() + 48h

value	String	all settings as text
-------	--------	-------------------------

```
// Stores: logoUrl, welcomeTitle(Ar),  
// welcomeSubtitle(Ar), primaryColor,  
// accentColor, *PageBg, smtpHost, etc.
```

API Endpoints

All endpoints are prefixed with `/api`. Admin routes require a JWT Bearer token obtained from the login endpoint. Public routes are open.

Method	Route	Auth	Description
GET	/schools	—	List active schools (includes hasAccessCode flag)
GET	/subjects?schoolId=	—	List active subjects for a school
GET	/teachers?subjectId=	—	List active teachers with coursePrice and discountPct
POST	/verify	—	Validate school code + student code → returns 4h JWT
POST	/codes/generate	verify JWT	Generate discount code; increments studentCode.useCount
GET	/codes/:codeId	—	Fetch code details with teacher/school/pricing info
POST	/codes/resend	—	Resend existing active code to student email
POST	/codes/:codeId/redeem	—	Mark code as USED (called by partner platform)
GET	/public/settings	—	Returns theme/branding settings for frontend rendering
POST	/admin/auth/login	—	Email + password → returns admin JWT
GET	/admin/schools	admin	List all schools
POST	/admin/schools	admin	Create school
PUT	/admin/schools/:id	admin	Update school
DELETE	/admin/schools/:id	admin	Delete school (cascades subjects + teachers)

Method	Route	Auth	Description
GET	/admin/access-codes/schools	admin	List schools with their access codes
POST	/admin/access-codes/schools/:id/generate	admin	Generate/regenerate school access code
GET	/admin/access-codes/students?schoollid=	admin	List student codes for a school
POST	/admin/access-codes/students/generate	admin	Bulk generate student codes (up to 500)
PUT	/admin/access-codes/students/:id	admin	Update label, maxUses, or isActive
DELETE	/admin/access-codes/students/:id	admin	Delete student code
GET	/admin/access-codes/students/export	admin	Export student codes as CSV
GET	/admin/reports	admin	Stats: codes/month, redemption rates, discount totals
GET	/admin/settings	admin	Get all platform settings
PUT	/admin/settings	admin	Save settings to DB (persists across restarts)
POST	/admin/upload	admin	Upload logo image (max 5MB, jpg/png/gif/webp/svg)
POST	/webhooks/partner-redemption	HMAC	Inbound webhook from partner platform (HMAC-SHA256 verified)
POST	/mock-partner/receive	—	Dev mock: logs incoming discount registration, returns 200

Discount Code Generation Flow

The code generation pipeline validates, calculates, stores, notifies, and integrates — all in a single request with async side-effects for email and partner notification.

1

Verify JWT Token

Extract `verificationToken` from request body. Decode JWT to get `schoolId` and `studentCodeId`. Reject if expired or `schoolId` mismatch.

2

Validate Entity Chain

Confirm `school` → `subject` → `teacher` chain is valid, all active, and the teacher belongs to the subject which belongs to the school.

3

Rate Limit Check

Max 3 codes per (`studentEmail` + `teacherId`) in 24h — checked in DB. Returns HTTP 429 with `Retry-After` header on violation.

4

Price Snapshot

Calculate: `originalPrice = teacher.coursePrice`, `finalPrice = originalPrice - round(originalPrice × discountPct / 100)`. Snapshot prevents price changes affecting existing codes.

5

Generate Unique Code

Format: `EDU-{INITIALS}-{YEAR}-{RANDOM6}` using uppercase alphanumeric. Collision-retry up to 5 times. Stored with `expiresAt = now() + 48h`.

6

Save to PostgreSQL + Redis Cache

Persist `DiscountCode` record. Cache in Redis: `SET code:{codeId} {status} EX 172800` (48h TTL) for fast status lookups.

7

Increment Student Code Usage

Atomically increment `studentCode.useCount`. If `useCount ≥ maxUses`, the next generation attempt will be rejected at the verify step.

8

Send Email (async)

Fire-and-forget via Nodemailer: HTML email with code, price breakdown, and expiry. Uses `SMTP_HOST/USER/PASS` from environment.

9

POST to Partner Webhook (async)

Sign payload with HMAC-SHA256 using `PARTNER_WEBHOOK_SECRET` . POST to `PARTNER_API_URL` with retry (3× exponential backoff). Logs to IntegrationLog table.

Security Model

Multiple layers of protection covering authentication, access control, abuse prevention, and data integrity.

Authentication

- Admin: JWT signed with HS256, stored in localStorage, sent as Bearer token
- Student: Short-lived 4h verification JWT issued after double-code check
- Webhook: HMAC-SHA256 signature on all outbound partner calls
- Inbound webhooks: HMAC signature verified before processing
- Passwords hashed with bcrypt (cost factor 10)

Access Control

- School-level access code — one per school, admin-generated
- Student-level code — unique per student, configurable max uses
- Both codes required before any discount code can be generated
- Schools without access codes allow open access (backward compatible)
- Admin middleware on all /admin/* routes via requireAuth()

Abuse Prevention

- IP-level rate limiting via express-rate-limit on all public routes
- Email + teacher combination limited to 3 codes per 24 hours
- Student code useCount checked atomically before each generation
- All codes expire automatically after 48 hours
- Unique constraint on code strings prevents duplicates

Data Protection

- All secrets in environment variables, never in code
- PostgreSQL and Redis not exposed outside Docker network
- File uploads restricted: max 5MB, image MIME types only
- Uploaded files stored in named Docker volume, served via /api/uploads
- Nginx adds X-Frame-Options, X-Content-Type-Options in production

 routes/verify.js — double verification logic

```
// Step 1: Check school access code
const school = await prisma.school.findUnique({ where: { id: schoolId } })
if (school.accessCode !== schoolCode.trim().toUpperCase())
```

```
    return res.status(403).json({ error: 'Invalid school code' })

    // Step 2: Check student code + use limit
    const sc = await prisma.studentCode.findFirst({
      where: { schoolId, code: studentCode.toUpperCase(), isActive: true }
    })
    if (!sc || sc.useCount ≥ sc.maxUses)
      return res.status(403).json({ error: 'Student code limit reached' })

    // Step 3: Issue short-lived verification token
    const token = jwt.sign(
      { schoolId, studentCodeId: sc.id },
      process.env.JWT_SECRET,
      { expiresIn: '4h' }
    )
```

Production Deployment

The entire platform ships as a single Docker Compose stack. One command brings up all services. SSL is handled by Nginx with Let's Encrypt certificates.

nginx/nginx.conf — routing rules

```
upstream backend { server backend:3001; }
upstream frontend { server frontend:80; }

server {
    listen 80;
    server_name edudiscount.com;

    # All API calls → Node.js backend
    location /api/ {
        proxy_pass http://backend;
        proxy_set_header X-Real-IP $remote_addr;
    }

    # All other requests → React SPA
    location / {
        proxy_pass http://frontend;
        try_files $uri $uri/ /index.html;
    }
}

# Production: adds SSL + security headers
# Strict-Transport-Security, X-Frame-Options,
# X-Content-Type-Options, gzip compression
```

.env — required environment variables

```
DATABASE_URL=postgresql://user:pass@postgres:5432/edudiscount
REDIS_URL=redis://redis:6379
JWT_SECRET=strong-random-secret-min-32-chars
ADMIN_SEED_EMAIL=admin@yourdomain.com
ADMIN_SEED_PASSWORD=StrongPassword@123
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your@gmail.com
```

```
SMTP_PASS=your-app-password  
PARTNER_API_URL=https://partner-platform.com/webhook  
PARTNER_WEBHOOK_SECRET=hmac-secret  
FRONTEND_URL=https://edudiscount.com
```

التوثيق التقني الشامل

البنية التقنية، قواعد البيانات، واجهات API، نموذج الأمان، ودليل النشر

Node.js 20

Express 4

PostgreSQL 15

Prisma ORM

Redis 7

React 18 + Vite

Tailwind CSS

Docker Compose

Nginx

JWT Auth

الإصدار	الخدمات	قاعدة البيانات	API نقاط
1.0.4	Docker حاويات 5	Prisma نماذج 6	مسار +30

بنية النظام

بنية تعتمد على الخدمات المصغرة وتعمل داخل Docker Compose. جميع حركة المرور الخارجية تدخل عبر Nginx، الذي يوجه طلبات API إلى الخادم الخلفي Node.js وباقي الطلبات إلى الواجهة الأمامية الثابتة React.



مخطط قاعدة البيانات

PostgreSQL مُدار عبر 6. Prisma ORM نماذج بعلاقات تسلسل حذف تلقائي. تُطبّق عمليات الترجيل تلقائياً عند بدء تشغيل الحاوية.

المدرسة - School نموذج

يمثل كل مدرسة مع رمز الوصول الخاص بها

accessCode	String?	رمز المدرسة - اختياري
nameAr	String?	الاسم بالعربية
isActive	Boolean	تفعيل/تعطيل المدرسة
studentCodes	StudentCode[]	حذف تسلسلي

كود الطالب - StudentCode نموذج

كود فريد لكل طالب مرتبط بمدرسته

code	String	STU-XXXXXX * فريد
label	String?	اسم/رقم الطالب للإدارة
maxUses	Int	الحد الأقصى للاستخدامات
useCount	Int	عدد مرات الاستخدام الفعلي
isActive	Boolean	تفعيل/إيقاف الكود

المدرس - Teacher نموذج

بيانات المدرس مع تفاصيل التسعير

coursePrice	Int	بالقروش = 100÷) جنيه)
discountPct	Int	% نسبة الخصم
nameAr	String?	الاسم بالعربية
platformTeacherId	String	المعرف في المنصة الخارجية

كود الخصم - DiscountCode نموذج

سجل كامل لكل كود خصم صدر

code	String	EDU-XXX-YYYY-ZZZ * فريد
originalPrice	Int	لغطة وقت الإصدار
finalPrice	Int	السعر بعد الخصم
status	String	ACTIVE USED EXPIRED
expiresAt	DateTime	الآن + 48 ساعة

نموذج الأمان المزدوج

طبقات متعددة من الحماية تشمل المصادقة، التحكم في الوصول، منع إساءة الاستخدام، وحماية البيانات.

🔒 الحماية من الاحتيال

- تحديد معدل الطلبات على مستوى IP عبر express-rate-limit
- الحد الأقصى: 3 أكواد لكل (بريد إلكتروني + مدرس) كل 24 ساعة
- فحص useCount ذري قبل كل إصدار للأكود
- جميع الأكواد تنتهي تلقائياً بعد 48 ساعة
- قيد التفرد على سلاسل الكود يمنع التكرار

🔑 نظام التوثيق المزدوج

- رمز المدرسة: كود عشوائي يولده المسؤول، يُوزع على الطلاب عبر القنوات الرسمية
- رمز الطالب: كود شخصي فريد لكل طالب بحد أقصى قابل للتعديل
- كلا الرمزین مطلوبان لإصدار أي كود خصم
- عند نجاح التحقق: يُصدر رمز JWT صالح 4 ساعات
- المدارس بدون رمز وصول: تسمح بوصول مفتوح (متوافق مع الإصدارات السابقة)

🔒 حماية البيانات

- جميع الأسرار في متغيرات البيئة، لا في الكود
- قاعدة البيانات و Redis غير مكشوفتين خارج شبكة Docker
- رفع الملفات مقيد: حجم 5MB، صور فقط
- Nginx يضيف رؤوس الأمان في بيئة الإنتاج

🔑 المصادقة

- مسؤول النظام: JWT موقع بـ HS256، يُرسل كـ Bearer token
- Webhook الخارجي: توقيع HMAC-SHA256 على جميع الطلبات الصادرة
- كلمات المرور مشفرة بـ bcrypt (معامل التكلفة 10)
- جميع مسارات /admin/ تتطلب requireAuth()

مرجع نقاط API

جميع النقاط مسبوقة بـ `api/`. مسارات المسؤول تتطلب رمز JWT. المسارات العامة مفتوحة.

الوصف	التوثيق	المسار	الطريقة
<code>hasAccessCode</code> قائمة المدارس النشطة مع علامة	-	<code>/schools</code>	GET
قائمة المواد لمدرسة معينة	-	<code>/subjects?schoolId=</code>	GET
قائمة المدرسين مع التسعير	-	<code>/teachers?subjectId=</code>	GET
لمدة 4 ساعات JWT التحقق المزدوج → إصدار	-	<code>/verify</code>	POST
إصدار كود خصم + زيادة عداد استخدام كود الطالب	verify JWT	<code>/codes/generate</code>	POST
استرجاع تفاصيل الكود مع بيانات المدرس والسعر	-	<code>/codes/:codeId</code>	GET
إعادة إرسال الكود النشط إلى البريد الإلكتروني	-	<code>/codes/resend</code>	POST
JWT تسجيل الدخول للمسؤول → إصدار	-	<code>/admin/auth/login</code>	POST
إنشاء/تجديد رمز وصول المدرسة	admin	<code>/admin/access-codes/schools/:id/generate</code>	POST
إنشاء أكواد طلاب بالجملة (حتى 500)	admin	<code>/admin/access-codes/students/generate</code>	POST
تحديث الحد الأقصى للاستخدامات أو تعطيل الكود	admin	<code>/admin/access-codes/students/:id</code>	PUT
CSV تصدير أكواد الطلاب كملف	admin	<code>/admin/access-codes/students/export</code>	GET
إحصائيات: الأكواد شهرياً، معدلات الاسترداد، إجمالي الخصومات	admin	<code>/admin/reports</code>	GET
حفظ إعدادات المنصة في قاعدة البيانات	admin	<code>/admin/settings</code>	PUT
(صور فقط 5MB، رفع الشعار)	admin	<code>/admin/upload</code>	POST

الوصف	التوثيق	المسار	الطريقة
Webhook (HMAC- SHA256) وارد من المنصة الشريكة	HMAC	/webhooks/partner-redemption	POST